ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

HPC Lab for CSE

FS 2026
Dr. R. Käppeli,
Dr. F. Gatti, J. Bachmann,
H. Schrader, A. Pietak,
S. Utzinger, V. Vogt,
B. Zemann.

Project 3

Due date: Monday 13 April 2026, 18:00 (Europe/Zurich). Please carefully review the information regarding the deadline on the course webpage. Follow the instructions in Section A.

In this project, we will develop a parallel PDE mini-app using `OpenMP`. `OpenMP` seems to be a straightforward way to parallelize (incrementally) a given application by simply adding directives to compute intensive loops. However, it turns out that getting a truly scalable `OpenMP` application is far from trivial in many cases, especially on today's CPUs featuring dozens of cores. The idea of this project is to confront you (in a friendly manner, of course) with this unfortunate truth.

For the whole project, the simulations must be run on the Euler (phase 2) nodes with AMD EPYC 7763 CPUs (for which we have privileged access during the lab hours), and we will use the default software stack using the GNU Compiler Collection:

```
user@eu-login-14:~$ module load stack gcc
user@eu-login-14:~$ gcc --version
gcc (Spack GCC) 12.2.0
Copyright (C) 2022 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

However, feel free to try and develop on other available systems (e.g., your workstation or laptop) and compilers, but please make sure to document them in your report if you include results.

You find all the skeleton source codes for the project on the course page.

While developing/debugging your code, it can be useful to work in an interactive session as follows:

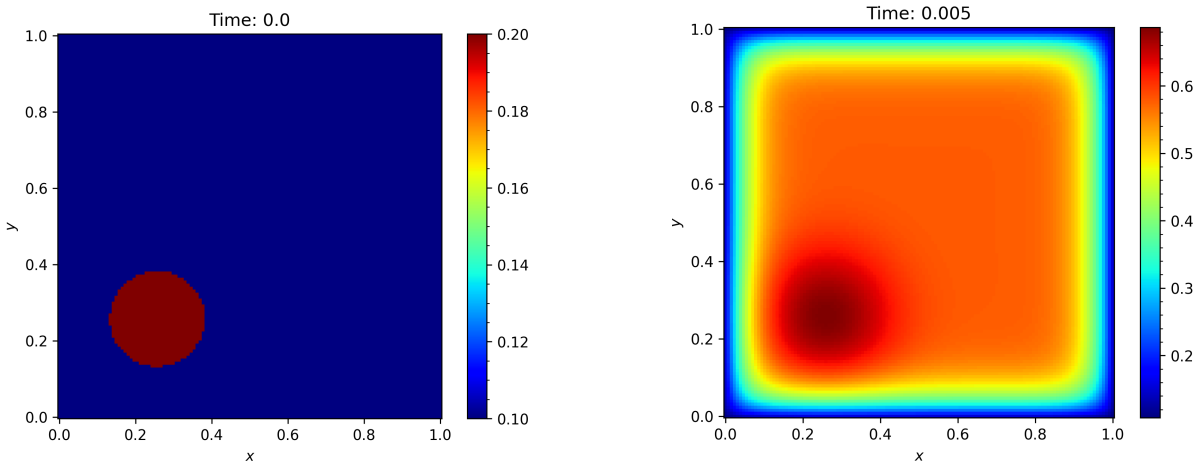
```
user@eu-login-14:~$ salloc --ntasks=1 --cpus-per-task=4 --mem-per-cpu=1024 \
                           --constraint=EPYC_7763                           \
                           --time=01:00:00 # allocate resources
salloc: Pending job allocation 59932397
salloc: job 59932397 queued and waiting for resources
salloc: job 59932397 has been allocated resources
salloc: Granted job allocation 59932397
salloc: Waiting for resource configuration
salloc: Nodes eu-a2p-410 are ready for job
user@eu-login-14:~$ srun --pty bash # run the interactive session
user@eu-a2p-410:~$ export OMP_NUM_THREADS=4 # set the number of threads
```

This allocates and runs an interactive session on an Euler VII (phase 2) node with up to 4 cores and 4GB of memory for one hour[1]. The following version using *short options* can also be useful:

```
user@eu-login-14:~$ salloc -n1 -c4 --mem-per-cpu=1024 -C EPYC_7763 -t 60
salloc: Pending job allocation 59933009
salloc: job 59933009 queued and waiting for resources
salloc: job 59933009 has been allocated resources
salloc: Granted job allocation 59933009
salloc: Waiting for resource configuration
salloc: Nodes eu-a2p-345 are ready for job
user@eu-login-14:~$ srun --pty bash
user@eu-a2p-345:~$
```

Even shorter, but with a warning (that you you can safely ignore)

[1] Please note that this is a two steps process: first it allocates the resources with `salloc` and then it runs the interactive session with `srun`.



(a) (b)

Figure 1: The population concentration at time $t = 0$ (left) and $t = 0.005$ (right).

```
user@eu-login-14:~$ srun -n1 -c4 --mem-per-cpu=1024 -C EPYC_7763 -t 60 --pty bash
srun: Job step's --cpus-per-task value exceeds that of job (4 > 1). Job step may never
↪  run.
srun: job 59934082 queued and waiting for resources
srun: job 59934082 has been allocated resources
user@eu-a2p-369:~$
```

The Fisher's equation as an example of a reaction-diffusion PDE

The simple `OpenMP` exercises such as the π-computation or the parallel Mandelbrot set are good examples for the basic understanding of `OpenMP` constructs, but due to their simplicity they are far away from practical applications. In this project, we are going to implement a parallel PDE mini-app that solves something more sophisticated, but where the code is nevertheless still relatively short and easy to understand. Our mini-app will solve a prototypical reaction-diffusion equation with the finite difference method. Although its simplicity, it is an example of so-called stencil-based kernels that constitute the core of many important scientific applications on block-structured grids, including numerical climate modeling, astrophysical, cosmological simulations and many more in between.

We consider Fisher's equation that can be used to simulate simple population dynamics. In two-dimensional Cartesian coordinates it is given by

$$\frac{\partial s}{\partial t} = D\left(\frac{\partial^2 s}{\partial x^2} + \frac{\partial^2 s}{\partial y^2}\right) + Rs(1-s), \tag{1}$$

where $s = s(x, y, t)$ is the population concentration, D is the diffusion constant and R is the reaction constant. The left hand side represents the rate of change of s over time. On the right hand side, the first term describes the diffusion of s in space and the second term describes the growth of the population.

We consider Eq. (1) on a square Cartesian domain $(x, y) \in \Omega = [0, 1]^2$ with Dirichlet boundary conditions

$$s(x, y, t) = 0.1 \quad \text{for } (x, y) \in \partial\Omega \tag{2}$$

and the initial conditions

$$s(x, y, t = 0) = \begin{cases} 0.2 & \text{for } (x - x_c)^2 + (y - y_c)^2 < r^2, \\ 0.1 & \text{elsewhere,} \end{cases} \tag{3}$$

where $x_c = y_c = \frac{1}{4}$ and $r = \frac{1}{8}$. The initial conditions and the population concentration at time $t_f = 0.005$ are displayed in Fig. 1.

To discretize the domain Ω, we introduce a uniform grid composed of $(n + 2) \times (n + 2)$ grid points. The grid points are denoted as (x_i, y_j), where $x_i = i\ h$ and $y_j = j\ h$ for $i, j = 0, 1, \ldots, n + 1$ and $h = \frac{1}{n+1}$ is the uniform grid spacing. Here, $x_0 = 0$ and $x_{n+1} = 1$ define the boundaries along the x-axis, and similarly, $y_0 = 0$ and $y_{n+1} = 1$ define the boundaries along the y-axis. The discretization

is shown in Fig. 2. Likewise, we discretize time into steps $k = 0, \dots, n_t$ of size $\Delta t = \frac{t_f}{n_t}$. We denote by $s_{i,j}^k$ the approximation of the population concentration at the grid point (x_i, y_j) at time step k (i.e., $s_{i,j}^k \approx s(x_i, y_j, t^k)$).

We use a second-order finite difference discretization to approximate the spatial derivatives of s for all inner grid points, i.e.,

$$\begin{aligned}\left(\frac{\partial^2 s}{\partial x^2} + \frac{\partial^2 s}{\partial y^2}\right)_{i,j} &\approx \frac{s_{i+1,j} - 2s_{i,j} + s_{i-1,j}}{h^2} + \frac{s_{i,j+1} - 2s_{i,j} + s_{i,j-1}}{h^2} \\ &= \frac{1}{h^2}(-4s_{i,j} + s_{i-1,j} + s_{i+1,j} + s_{i,j-1} + s_{i,j+1}),\end{aligned} \tag{4}$$

for all grid point $(i, j) \in \{1, \dots, n\}$. This results in the so-called 5-point stencil and it is displayed in Fig. 2. Note that one distinguishes interior grid points, where we seek an approximate solution, and boundary grid points, which are fixed by the Dirichlet condition. In order to approximate the time derivative, we use a first-order implicit Euler difference scheme, which at time step k gives

$$\left(\frac{\partial s}{\partial t}\right)_{i,j}^k \approx \frac{1}{\Delta t}(s_{i,j}^k - s_{i,j}^{k-1}). \tag{5}$$

Putting together the components from Eqs. (4) and (5), we obtain the following discretization of Eq. (1):

$$\frac{1}{\Delta t}(s_{i,j}^k - s_{i,j}^{k-1}) = \frac{D}{h^2}\left(-4s_{i,j}^k + s_{i-1,j}^k + s_{i+1,j}^k + s_{i,j-1}^k + s_{i,j+1}^k\right) + Rs_{i,j}^k(1 - s_{i,j}^k), \tag{6}$$

which we will attempt to solve in order to obtain an approximate solution for the population concentration s. By moving all terms to the right-hand side and multiplying with h^2/D, we can reformulate Eq. (6) as

$$f_{i,j}^k = \left[-(4 + \alpha)s_{i,j}^k + s_{i-1,j}^k + s_{i+1,j}^k + s_{i,j-1}^k + s_{i,j+1}^k + \beta s_{i,j}^k(1 - s_{i,j}^k)\right] + \alpha s_{i,j}^{k-1} = 0 \tag{7}$$

for each grid point (i, j) and time step k with $\alpha = h^2/(D\Delta t)$ and $\beta = Rh^2/D$.

At time step $k = 1$, we initialize $s_{i,j}^{k-1} = s_{i,j}^0 = s(x, y, t = 0)$, and we look for approximate values $s_{i,j}^k$ that fulfill Eq. (7). For all (i, j) at a fixed k, we obtain a system of $N = n^2$ equations. We can see that each equation is quadratic in $s_{i,j}^k$, and therefore nonlinear which makes the problem more difficult to solve. We tackle this problem using Newton's method with which we iteratively try to find better approximations of the solution of Eq. (7). In order to formulate the Newton iteration, we introduce the following notation: Let $\mathbf{s}^k = [s_{1,1}^k, \dots, s_{n,1}^k, s_{1,2}^k, \dots, s_{n,n}^k]^T \in \mathbb{R}^N$ be a vector containing the approximate solution at time step k. The grid unknowns are arranged in the so-called *lexicographic ordering*, where the x-index runs fastest and the rows are stacked along the y-direction. Then, we can consider the set of equations $f_{i,j}^k$ as functions depending on $\mathbf{s}^k$ and define $\mathbf{f}(\mathbf{s}^k) = [f_{1,1}^k, \dots, f_{n,1}^k, f_{1,2}^k, \dots, f_{n,n}^k]^T \in \mathbb{R}^N$. For Newton's method, we then have at the l-th iteration

$$\mathbf{y}^{(l+1)} = \mathbf{y}^{(l)} - [\mathbf{J_f}(\mathbf{y}^{(l)})]^{-1}\mathbf{f}(\mathbf{y}^{(l)}), \tag{8}$$

where $\mathbf{J_f}(\mathbf{y}^{(l)}) \in \mathbb{R}^{N \times N}$ is the Jacobian of $\mathbf{f}$. We start with the initial guess $\mathbf{y}^{(0)} = \mathbf{s}^{k-1}$. However, for each iteration the inverse of the Jacobian $[\mathbf{J_f}(\mathbf{y}^{(l)})]^{-1}$ is not readily available. We do not compute it directly but instead use a matrix-free Conjugate Gradient (CG; see, e.g., [1, Sec. 7.4]) solver that solves the following linear system of equations for $\delta\mathbf{y}^{(l+1)} = \mathbf{y}^{(l+1)} - \mathbf{y}^{(l)}$:

$$[\mathbf{J_f}(\mathbf{y}^{(l)})]\delta\mathbf{y}^{(l+1)} = -\mathbf{f}(\mathbf{y}^{(l)}) \tag{9}$$

We iterate over l in Eq. (8) until a stopping criterion is reached and we obtain a final solution $\mathbf{y}^{\text{fin}}$. This is the approximate solution for time step k, i.e. $\mathbf{s}^k = \mathbf{y}^{\text{fin}}$ that we originally set out to find in Eq. (7). It is then in turn used as the initial guess to compute an approximate solution for the next time step $k+1$.

Code Walkthrough

The provided code on the course page for the project already contains most of the functionalities described above, a brief overview of the code is presented in this section. The main task of this project will be (i) to complete some parts of the sequential code and (ii) the parallelization of the code using `OpenMP`. This project will also serve as an example for using the message-passing interface MPI and GPU programming in subsequent projects. There are three files of main interest:

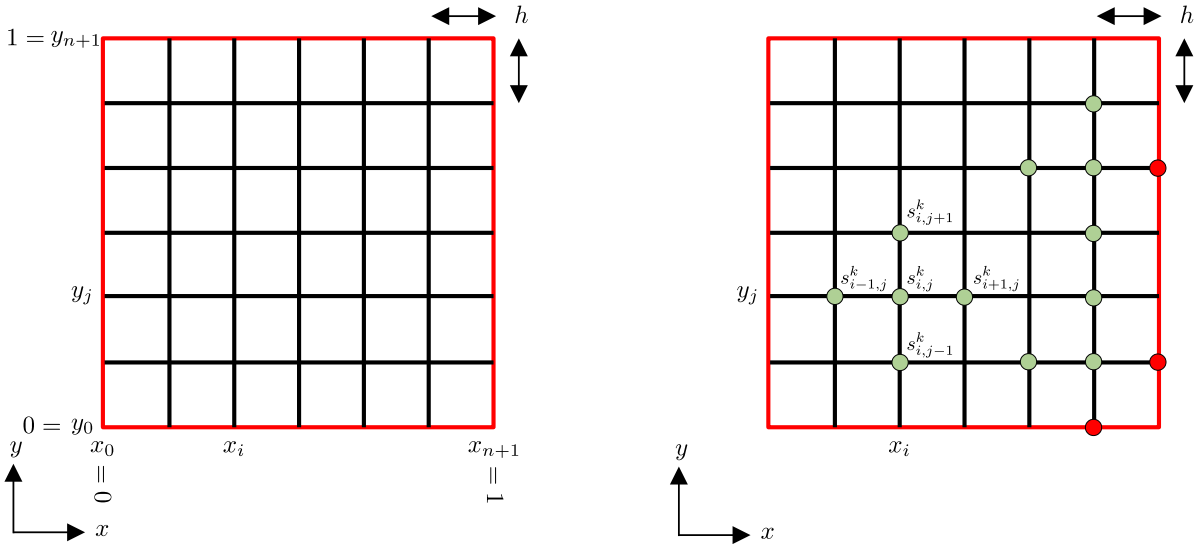


Figure 2: The left panel shows the discretization of the domain $\Omega = [0,1]^2$. The right panel shows the grid points involved in the computation of $f^k_{i,j}$ in (7): inner grid point (left), boundary grid point (upper right) and corner grid point (lower right).

- `main.cpp`: initialization and main time stepping loop.
- `linalg.cpp`: the BLAS level 1 (vector-vector) kernels and conjugate gradient solver. All the kernels of interest start with `hpc_xxxxx`.
  - Scalar product $x \cdot y$: `hpc_dot`.
  - Linear combination $z = \text{alpha} * x + \text{beta} * y$: `hpc_lcomb`.
  - The CG solver is already fully implemented in `hpc_cg`.
  - ...
- `operators.cpp`: the stencil operator for the finite difference discretization.

Compile and run the PDE mini-app on the Euler cluster

Use the `Makefile` to compile the mini-app:

```
user@eu-login-14:~$ module load stack gcc
user@eu-login-14:~$ make
```

Run the application on a compute node with selected parameters, e.g., domain size 128×128, 100 time steps and simulation time from $t = 0$ until $t = 0.005$:

```
user@eu-login-14:~$ srun -t 60 -C EPYC_7763 --pty bash
srun: job 59940317 queued and waiting for resources
srun: job 59940317 has been allocated resources
user@eu-a2p-533:~$ ./main 128 100 0.005
```

After you implement the first part of the assignment, the output of the mini-app should look like this:

```
user@eu-a2p-533:~$ ./main 128 100 0.005
================================================================================
                       Welcome to mini-stencil!
version   :: C++ Serial
mesh      :: 128 * 128 dx = 0.00787402
time      :: 100 time steps from 0 .. 0.005
iteration :: CG 300, Newton 50, tolerance 1e-06
================================================================================
```

```
--------------------------------------------------------------------------------
simulation took 0.171042 seconds
1513 conjugate gradient iterations, at rate of 8845.77 iters/second
300 Newton iterations
--------------------------------------------------------------------------------
### 1, 128, 100, 1513, 300, 0.171042 ###
Goodbye!
```

Please notice the penultimate line (starting with `###`), which can be useful for processing the output with a script. The mini-app produces `output.bov/bin` that contain the population concentration at final time, which can be visualized with

```
user@eu-a2p-533:~$ module load python # assuming stack and gcc already loaded
user@eu-a2p-533:~$ python plot.py # you might have to set up a suitable
                                  # environment, see below
```

This creates `output.png` showing the population concentration at final time. It should look like Fig. 1b.

In order to setup your Python environment, we recommend to create a virtual environment and install the necessary dependencies with

```
user@eu-a2p-533:~$ module load stack gcc python eth_proxy # eth_proxy is only
                                                          # necessary if you
                                                          # are on a compute
                                                          # node
user@eu-a2p-533:~$ python -m venv venv
user@eu-a2p-533:~$ . venv/bin/activate
user@eu-a2p-533:~$ pip install -r requirements.txt
```

1 Numerical scheme description and implementation of the linear algebra functions and the stencil operators [50 Points]

The provided implementation is a serial version of the PDE mini-app with some code missing. Familiarize yourself with the solved initial boundary value problem and numerical scheme. In your report, concisely summarize the problem and numerical method in the style of a scientific paper. Implement the missing code to get a working PDE mini-app:

- Implement the functions `hpc_xxxxx()` in `linalg.cpp`. Follow the comments in the code as they are there to help you with the implementation.
- Implement the missing stencil kernel in `operators.cpp`.

After completion of the above steps, the mini-app should produce correct results. Compare the number of conjugate gradient iterations and the Newton iterations with the reference output above. If the numbers are about the same, you have probably implemented everything correctly.

- Plot the solution at initial and final time with the script `plot.py` and include it in your report. It should look like in Figure 1.

2 Adding `OpenMP` to the nonlinear PDE mini-app [50 Points]

Once the serial version of the mini-app is working, we add `OpenMP` directives. In this project, you will measure and improve the scalability of your PDE simulation code. Scalability is the changing performance of a parallel program as it utilizes more computing resources. In particular, we are interested in a performance analysis of both **strong scalability** and **weak scalability**. Strong scaling is identifying how a parallel PDE solver gets faster for a fixed PDE problem size. That is, we have the same discretization points per direction, but we run it with more and more processing units (i.e., `OpenMP` threads) and we expect that it will compute its result faster. Weak scaling speaks to the latter point: how effectively can we increase the size of the problem we are dealing with? In a weak scaling study, each processing unit has the same amount of work, which means that running on more processing units increases the total size of the PDE simulation.

2.1 Welcome message in `main.cpp` and serial compilation [5 Points]

Replace the welcome message in `main.cpp` with a message that informs the user about (i) that the code is using `OpenMP` and (ii) the number of threads:

```
user@eu-login-14:~$ salloc -n1 -c4 --mem-per-cpu=1024 -C EPYC_7763 -t 60
salloc: Pending job allocation 59959784
salloc: job 59959784 queued and waiting for resources
salloc: job 59959784 has been allocated resources
salloc: Granted job allocation 59959784
salloc: Waiting for resource configuration
salloc: Nodes eu-a2p-296 are ready for job
user@eu-login-14:~$ srun --pty bash
user@eu-a2p-296:~$ export OMP_NUM_THREADS=4
user@eu-a2p-296:~$ ./main 128 100 0.005
================================================================================
                      Welcome to mini-stencil!
version   :: C++ OpenMP
threads   :: 4
mesh      :: 128 * 128 dx = 0.00787402
time      :: 100 time steps from 0 .. 0.005
iteration :: CG 300, Newton 50, tolerance 1e-06
================================================================================
--------------------------------------------------------------------------------
simulation took 0.114456 seconds
1515 conjugate gradient iterations, at rate of 13236.6 iters/second
300 Newton iterations
--------------------------------------------------------------------------------
### 4, 128, 100, 1515, 300, 0.114456 ###
Goodbye!
```

Make sure that the code still compiles without `OpenMP` enabled and reports the serial message.
Hint: Assume that the supported compilers feature the `_OPENMP macro` specifications[2].

2.2 Linear algebra kernel [10 Points]

Add `OpenMP` directives to parallelize the computationally intensive loops in the functions `hpc_XXXX()`.

2.3 The diffusion stencil [15 Points]

Add `OpenMP` directives to parallelize the stencil operator in `operators.cpp`.

2.4 Strong scaling [10 Points]

How does your code scale for different resolutions? Plot the time to solution for $N_{\text{CPU}} = 1, 2, 4, 8, 16$ threads across resolutions of $n \times n$, where $n = 64, 128, 256, 512, 1024$. Critically interpret the results in your report.
For example: For a resolution $n = 64$, plot the time to solution for 64×64 on $N_{\text{CPU}} = 1, 2, 4, 8, 16$ threads.

2.5 Weak scaling [10 Points]

How does code scale for constant work by threads ratio? Plot the time to solution as the total problem size and the number of threads $N_{\text{CPU}} = 1, 4, 16, 64$ increase proportionally, to maintain a constant workload per thread, and the base resolutions $n \times n$ and $n = 64, 128, 256$. Critically interpret the results in your report.
For example: For base resolution $n = 64$, plot the time to solution for 64×64 on $N_{\text{CPU}} = 1$, 128×128 on $N_{\text{CPU}} = 4$, 256×256 on $N_{\text{CPU}} = 16$ and 512×512 on $N_{\text{CPU}} = 64$.
Hint: Keep in mind that the convergence of the nonlinear solver might depend on the resolution.

[2] https://www.openmp.org/spec-html/5.2/openmpse16.html#x47-460003.3

A Additional notes and submission details

Submit your **report** and **all source code files**, including your build files (e.g., `Makefile`(s)) and other scripts (e.g., batch job scripts, plotting scripts), as a compressed tar archive. Your LaTeX report must be based on the provided template from the course webpage and should summarize your results and observations for all sections. Your submission should be uploaded to SAM-UP and must include

- Your report, named as `project_number_lastname_firstname.pdf`. You may discuss all questions with anyone you like. However, (i) you must list all collaborators in your submission, and (ii) you must prepare and write up the submission independently.
- A compressed tar[3] archive, named `project_number_lastname_firstname.tar.gz`, containing:
  - All source code files.
  - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
  - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
- The total size of your submission (report PDF and archive) must not exceed 25 MB.
- File names must contain only `ASCII` characters (avoid spaces, special symbols, and non-English letters).

Please strictly follow these instructions and naming conventions. Failure to do so results in additional work for the teaching assistants.

References

[1] Uri M. Ascher and Chen Greif. *A First Course in Numerical Methods*. Society for Industrial & Applied Mathematics (SIAM), June 2011. doi:10.1137/9780898719987.

[3] https://en.wikipedia.org/wiki/Tar_(computing)